
CS3460: Data Structures

Final Exam

Total Points: 100

Problems

Eat at Manny's (40 points) **3**

New Neighbors (60 points) **4**

Notes on Grading: Grading will be handled via Gradescope. Any file you submit must begin with the following package statement to be accepted:

```
1 package student;
```

Your code will be compiled and run against a series of tests. You may submit as many times as you would like before the assignment closes. To earn full credit, you must submit to both Gradescope and AsULearn before the assignment deadline.

Important Note: You are not allowed to use any classes or code from the Java Collections library (built-in Java data structures). The purpose of these assignments is to build these data structures from first principles. Programs which use any of these libraries will receive zero points. This only applies to Collections, and does not include classes like **Scanner** or other utility classes.

Submission: Please submit `Islands.java` and `Heights.java`.

Honor Code

The following document is an individual exam meant to be worked on and completed by an individual student without the aid of any other student or outside references. Discussing the contents of this exam with another student or exchanging material of any kind with another student constitutes plagiarism. Please review Appalachian State University's guidelines on this matter.

By submitting this exam, you implicitly agree to uphold the honor code. You may use any code posted on AsULearn, any notes you have taken, and any labs you have submitted as reference material, as well as references online for general Java programming help (syntax, semantics, and standard library documentation). You may not search the Internet for specific solutions to problems.

In preparation for this exam, I have found a wide variety of code across the internet that seeks to solve similar types of problems, and I will be using MOSS, a tool for measuring software similarity, to help identify cases of plagiarism.

1. **Eat at Manny’s (40 points):** You’ve come up with an exciting new idea in the world of fast food! People keep getting stranded on desert islands, and you’re going to finally take advantage of this hungry untapped market by building fast food restaurants on every island you can find – though it is still unclear how the employees are going to get to work.

A map is provided to you in a two-dimensional square $n \times n$ grid. Each grid cell either contains the period character `.` (water) or the asterisk character `*` (land). Two land tiles are considered connected if they are adjacent to one another in any of the standard eight directions (horizontal, vertical, or diagonal). An “island” is considered to be all land tiles reachable from one another by a path of connected land tiles. You may consider the map to be surrounded on all sides by water. Please write a program `Islands.java` containing a method

```
1 public static int countIslands(String filename)
```

that reads a file containing a map (like the one described below) and returns the number of islands.

The input contains an integer $n \leq 1000$ on the first line, representing the size of the grid. The following n lines describe the map, with each line containing n characters with dots and stars representing water and land respectively. Your program should only output a single integer representing the number of islands.

map-sample.txt

```
5
...*.
**.*
*...
*.*.*
**...
```

Output:

3

2. **New Neighbors (60 points):** You run the construction permit office in a small, historic city. As the city has grown, the city has tried to push back on the construction of high-rise buildings. They have come up with a new strategy: the cost of obtaining a permit will be based not on the height of the building being constructed, but on the height of the next-tallest already-built building (the people running the permit office say that this is in the interest of fairness for buildings that were already allowed to be built).

For instance, assume there are already buildings in town with heights [5, 2, 17, 9, 3, 6, 10, 14, 11, 7].

If a client wants to build a building of height $h = 16$, then the cost of the permit will be \$14, because the next tallest building has height $h = 14$. Specifically, it is based on the tallest value that is strictly less than the height requested. This matters for when a permit is requested for a building whose height is exactly equal to the height of another building. If no such building exists, then $h = 0$ (the permit is free!).

Each permit includes how tall they plan to build their building. Construction in this town is uncommonly fast, and requested permits will impact future permits. Please write a program `Heights.java` that accepts a list of existing buildings and a list of permits, and returns the sum of the costs of all requested permits.

```
1 public static int permitCost(int[] heights, int[] requests)
```

You have compiled a list of all the high-rise buildings in the city. There are n buildings and q permit requests, where $1 \leq n, q \leq 10000$ (so efficiency is key!). You want to compute the total cost of all requested permits. The permit office is depending on you to help them determine their annual revenue, and they need their answer as soon as possible.